CSE-4355: Electromechanical Systems and Sensors
Lab #3: PMDC Motor with Back-EMF Speed Sensing - Report

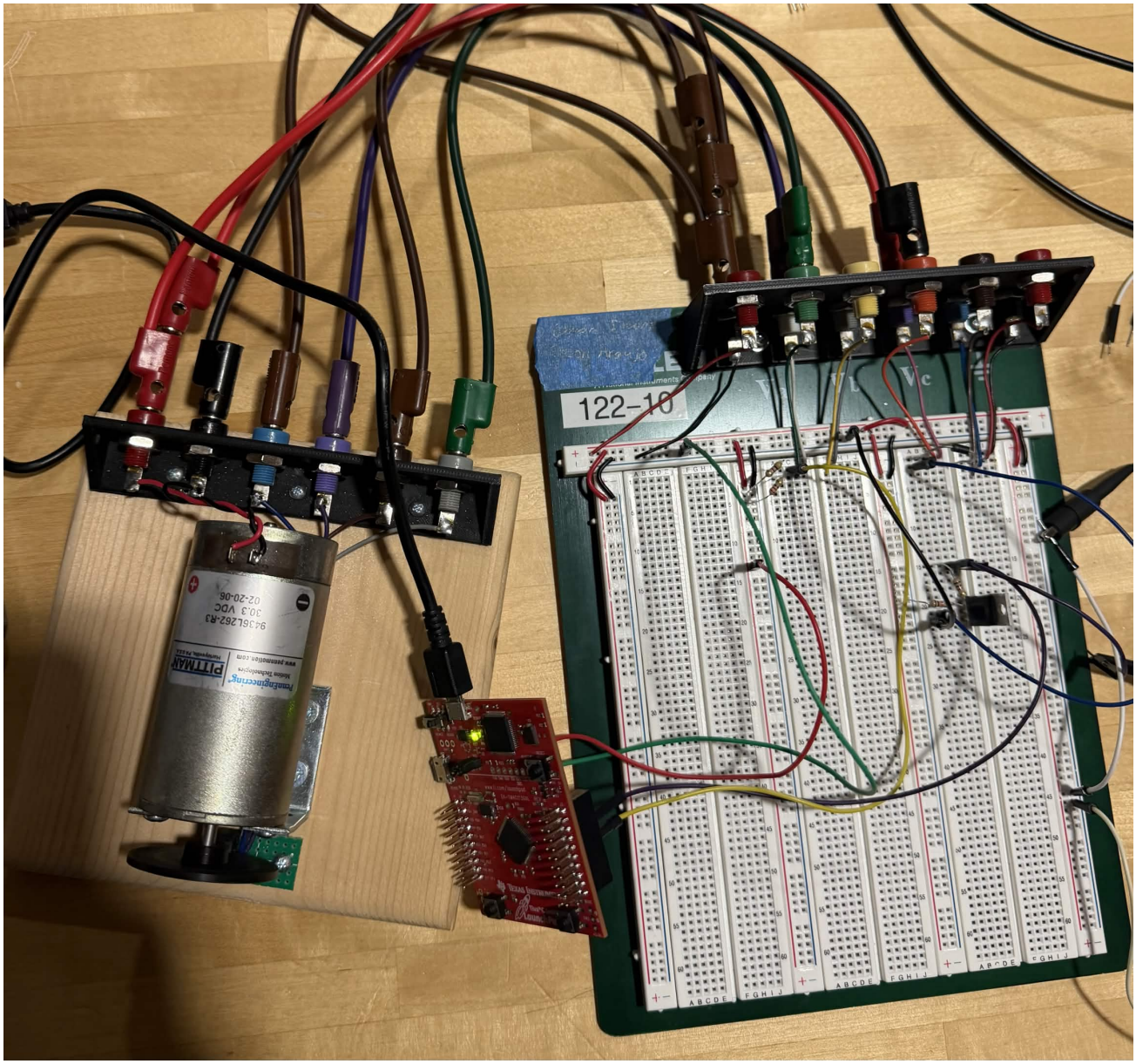


Figure 1: Completed Lab Setup

Subhaan Elburz
ID: 1002135522
sme5522@mavs.uta.edu
Department of Computer Science & Engineering

October 11, 2026

1 Introduction

This lab was about controlling and measuring the speed of a permanent magnet DC (PMDC) motor with the TM4C. We drove the motor with PWM through a MOSFET and measured its speed with an optical switch. We then measured the speed a second way from the motor's back-EMF, calibrated that reading against the optical speed, and found the range where it is accurate.

2 Equipment and Setup

Equipment:

- PMDC motor with a slotted disc (32 optical pulses per revolution)
- FQP20N06L n-channel MOSFET and a 1N581x Schottky flyback diode
- Optical slotted switch, two 1N914 diodes, and 220 ohm, 4.7k ohm, 47k ohm, and 10k ohm resistors
- TM4C, breadboard, 10 V bench supply, multimeter, and oscilloscope

Wiring: All circuit connections can be seen in the circuit diagram in Figure 2. The specific microcontroller pins used were: PC4 for the PWM output to the MOSFET gate, PC6 for the optical switch, PE0 for the back-EMF input, and the on-board push buttons PB0 (PF4) and PB1 (PF0).

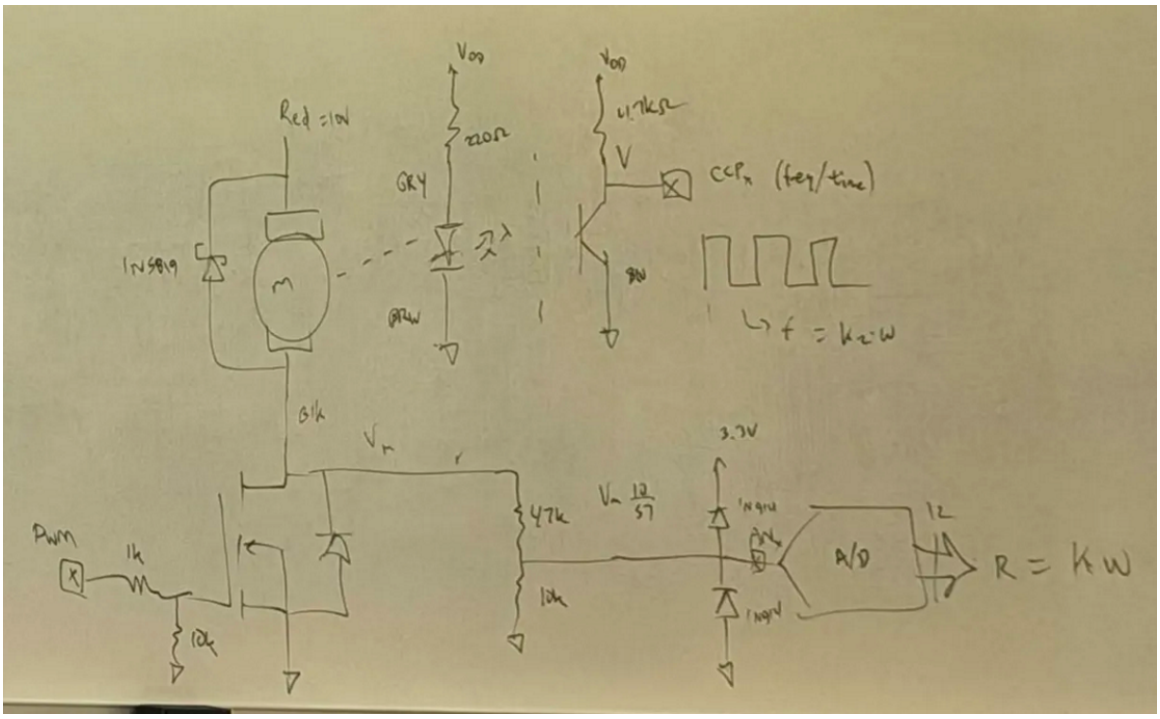


Figure 2: Complete circuit diagram

3 Motor Drive and Button Control (Steps 1 through 4)

Observations: The motor started at 50 percent duty cycle, and PB1 and PB0 raised and lowered it by 10 percent. Figure 3 shows the PWM at 20, 50, and 80 percent from left to right. The motor sped up as the duty cycle increased, and it did not run at 0 or 10 percent.



Figure 3: PWM output at 20, 50, and 80 percent duty cycle (left to right)

Concepts: The average voltage across the motor is roughly the duty cycle times the supply, so a higher duty cycle makes the motor spin faster. At 10 percent, that voltage was not high enough to overcome friction, so the motor did not run. The flyback diode protects the MOSFET from the voltage spike when the motor turns off.

How Code Works: The duty cycle is stored as a number from 0 to 1000, and each button press adds or subtracts 100. The buttons use interrupts (PF0 had to be unlocked), and a 1 kHz timer turns each interrupt off for 200 ms so one press only counts once. At 0 percent the PWM function sets the output low directly, because otherwise the gate would stay high and the motor would run at full speed.

4 Optical Speed Measurement (Step 5)

Observations: To know how fast the motor was actually spinning, we used the optical switch and printed the speed once per second. Figures 4 and 5 show the output after stepping down to 10 percent and up to 30 percent. These were captured with the back-EMF code turned off (see Section 5), so only the optical values appear.

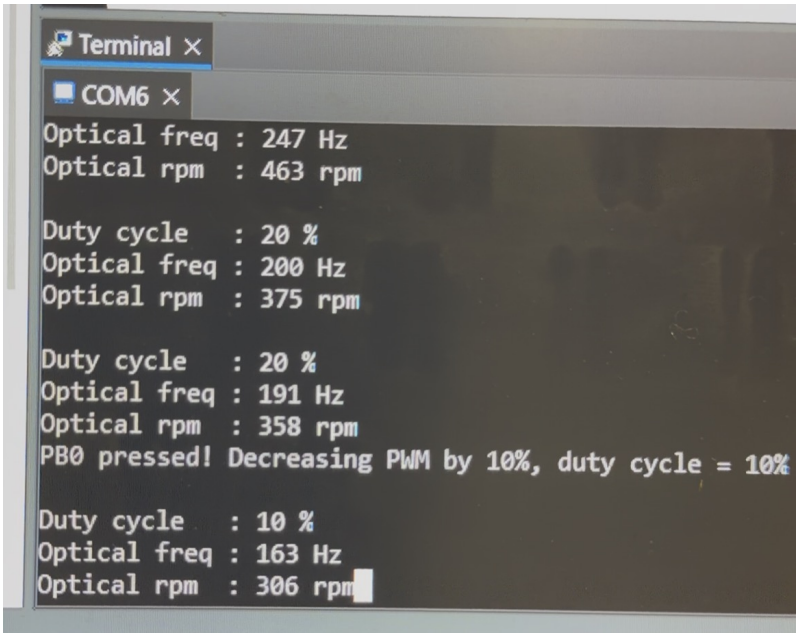


Figure 4: UART output after PB0 was pressed (20 percent to 10 percent duty cycle)

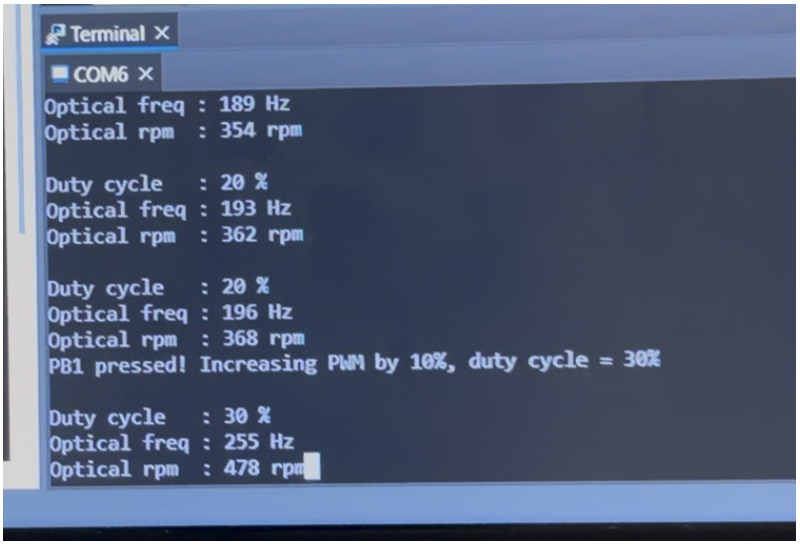


Figure 5: UART output after PB1 was pressed (20 percent to 30 percent duty cycle)

Concepts: Counting pulses for exactly one second gives the frequency directly, and 32 pulses make one revolution:

$$\text{rpm} = \frac{f \cdot 60}{32}, \qquad \text{for example (from figure 5):} \qquad \frac{(193)(60)}{32} \approx 362 \text{ rpm}$$

How Code Works: Wide Timer 1 counts the pulses coming into PC6, and Timer 1 interrupts once per second. In that interrupt, the code saves the count as the frequency, resets the count to 0, and sets a flag so the main loop prints the speed.

5 Back-EMF Sensing (Steps 6 and 7)

Observations: The second speed measurement uses the motor's back-EMF. Every 20 ms, the controller turned the PWM off, waited for 800 μs, read the drain voltage with the ADC, and turned the PWM back on. Figure 6 is a screenshot of one print from our data text file, where the UART output was copy/pasted into. In the calibration sweep (Section 6), the average drain voltage printed was 9.72 V at 0 percent duty cycle and 0.38 V at 100 percent.

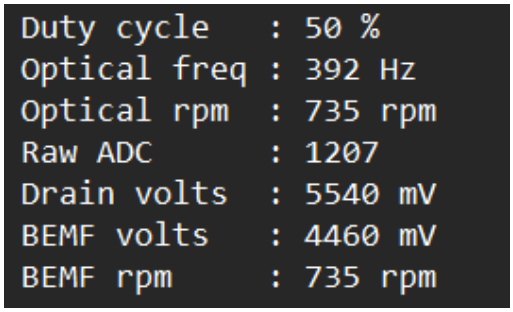


Figure 6: UART output at 50 percent duty cycle with both speed measurements

Concepts: A spinning PMDC motor acts as a generator, so its back-EMF grows with speed. Once the MOSFET is off and the leftover motor current has died out, the drain voltage is the supply minus the back-EMF, so a faster motor gives a lower drain voltage. We set the wait for the current to die out to 800 μs, raising it from 500 μs until

the reading with the motor not running at 10 percent matched the reading at 0 percent. The motor is only off for about 0.8 ms of every 20 ms, so its speed barely changes.

The ADC is 12 bit, so its 4096 counts cover 0 to 3300 mV. The 47k ohm and 10k ohm divider only lets 10/57 of the drain voltage reach the pin, so the drain voltage is 57/10 times the pin voltage. The back-EMF is the 10 V supply minus the drain voltage. For the print in Figure 6 (Raw ADC of 1207):

$$V_{pin} = \frac{1207 \cdot 3300}{4096} = 972 \text{ mV}, \qquad V_{drain} = 972 \cdot \frac{57}{10} = 5540 \text{ mV}, \qquad V_{BEMF} = 10000 - 5540 = 4460 \text{ mV}$$

How Code Works: Timer 2 fires at 50 Hz, and its interrupt (Figure 7) turns the PWM off, waits 800 μs, reads the ADC, and turns the PWM back on. This takes about 0.8 ms, which is under the 1 ms requirement.

All of the back-EMF code is wrapped in `PART2`, a macro defined in the header file. If it is defined, the back-EMF code and its UART prints are included. If it is commented out, they are left out and only the PWM and optical measurement run.

```
132 void timer2_handler(void)
133 {
134     TIMER2_ICR_R = TIMER_ICR_TATOCINT;  // first clear the interrupt
135
136     // turn off PWM, wait for it to settle, read the ADC value, and turn PWM back on
137 #ifdef PART2
138     set_m0pwm6_duty_cycle(0);
139     waitMicrosecond(PWM_WAIT);
140     bemf_raw = (uint32_t) readAdc0Ss3();
141     set_m0pwm6_duty_cycle(duty_cycle);
142 #endif
143 }
```

Figure 7: Back-EMF sampling interrupt (Timer 2, 50 Hz)

6 Back-EMF to RPM Correlation (Step 8)

Observations: To turn the ADC reading into rpm, we stepped the duty cycle from 0 to 100 percent and recorded five prints at each step. Table 1 shows the optical rpm and raw ADC reading for 20 to 100 percent, where each value is the average of the last four prints. The first print after each button press was left out because its one-second optical window still includes the old speed, and 0 and 10 percent were left out because the motor was not running.

Duty cycle	Optical rpm	Raw ADC
20%	228.2	1845.2
30%	398.0	1646.2
40%	565.2	1409.5
50%	730.0	1190.0
60%	900.5	993.2
70%	1073.0	808.5
80%	1236.0	625.8
90%	1409.0	344.2
100%	1578.5	84.2

Table 1: Calibration sweep averages (before calibration)

As the speed rose, the raw reading dropped in a nearly straight line. We used the slope and intercept functions in Excel on the nine averages in Table 1, with the optical rpm as y and the raw ADC as x, which gave the line

$$\text{rpm} = -0.7823 \cdot \text{raw} + 1679.7, \qquad R^2 = 0.9973$$

The slope of -0.7823 means the speed drops 0.7823 rpm for every count the raw reading goes up, and the intercept of 1679.7 is the rpm where the line crosses a raw reading of 0. An R^2 close to 1 means the nine points sit almost exactly on the line.

Concepts: The slope is negative because the raw reading goes down as the speed goes up. To use the line, we rewrote it in terms of how far the reading has dropped from its stopped value, which is the raw reading where the line reaches 0 rpm. Setting rpm to 0 gives that value:

$$0 = -0.7823 \cdot \text{raw} + 1679.7 \quad \rightarrow \quad \text{stopped_raw} = \frac{1679.7}{0.7823} = 2147$$

The slope with its sign flipped is the rpm for each count the reading drops, so the final equation is

$$\text{rpm_per_count} = 0.782, \qquad \text{rpm} = (2147 - \text{raw}) \cdot 0.782$$

For the print in Figure 6, $(2147 - 1207)(0.782) = 735$ rpm, which matches the optical rpm. The stopped value of 2147 is 29 counts above the reading at rest of 2118 (the average of the five prints at 0 percent), because the line was only fit to the points where the motor was running. This is why a stopped motor reads about 25 rpm instead of 0.

How Code Works: The two constants are in the back-EMF rpm function (Figure 8), which subtracts the raw reading from the stopped value and multiplies by the slope. It uses decimal (floating-point) numbers because the stored raw value cannot be negative. If noise pushes the reading above the stopped value, the result is set to 0 rpm.

```
188 uint32_t motor_get_bemf_rpm(void)
189 {
190     float stopped_raw = 2147.0f;        // raw ADC reading when motor is at 0 rpm
191     float rpm_per_count = 0.782f;       // rpm for every 1 ADC count
192
193     // save the raw adc value
194     float raw_adc = (float) bemf_raw;
195
196     // how many counts the reading has dropped since the motor was stopped
197     float bemf_counts = stopped_raw - raw_adc;
198
199     // if it is negative, clamp it to zero
200     if (bemf_counts < 0.0f)
201     {
202         bemf_counts = 0.0f;
203     }
204
205     // convert the back emf counts to rpm
206     float calculated_rpm = bemf_counts * rpm_per_count;
207
208     // round to the nearest whole rpm
209     return (uint32_t) (calculated_rpm + 0.5f);
210 }
211 #endif
```

Figure 8: Back-EMF speed calculation using the fitted constants

7 Validation and Valid Range (Steps 9 and 10)

Observations: With the constants loaded, the UART printed both speeds together (Figure 6). We repeated the sweep with five prints at each duty cycle, again leaving out the first print after each button press. For each duty cycle, we averaged the last four optical and back-EMF readings and calculated the error as the back-EMF rpm minus the optical rpm, divided by the optical rpm. Table 2 shows that the averages agree within 2 percent from 20 to 100 percent duty cycle.

Duty cycle	Optical rpm	BEMF rpm	Error
20%	218.0	217.0	-0.5%
30%	386.8	391.8	$+1.3\%$
40%	556.5	547.5	-1.6%
50%	735.0	741.8	$+0.9\%$
60%	909.2	910.2	$+0.1\%$
70%	1076.5	1058.0	-1.7%
80%	1228.5	1228.0	0.0%
90%	1399.5	1398.8	-0.1%
100%	1560.0	1530.2	-1.9%

Table 2: Back-EMF speed compared to optical speed (after calibration)

Single prints vary more than the averages. The ADC reading is taken at one instant, while the optical rpm is an average over one second. We also checked the error of each of the 36 single prints (four at each of the nine duty cycles from 20 to 100 percent). The average error was 3.3 percent; 30 of the 36 prints (83 percent) were within 5 percent, 35 of the 36 (97 percent) were within 10 percent, and the worst was 11 percent at 20 percent duty cycle.

Below 20 percent, the estimate does not work. At 0 percent, a stopped motor reads about 25 rpm, from the offset described in Section 6. At 10 percent, the motor was not running properly, so the optical readings jumped between 0, 203, 23, 9, and 17 rpm, and the back-EMF readings did not follow them.

Concepts: At low speed, the back-EMF is only a few ADC counts above the noise, so the estimate is not reliable. The back-EMF speed calculation is therefore valid from approximately 220 rpm to 1560 rpm (20 to 100 percent duty cycle).

8 Conclusion

In conclusion, we controlled the speed of a PMDC motor with a PWM-driven MOSFET and measured it with an optical switch. Sampling the drain voltage while the motor was briefly off gave its back-EMF, and fitting that to the optical speed gave $\text{rpm} = (2147 - \text{raw}) \cdot 0.782$. This matched the optical speed within 2 percent on average from about 220 rpm to 1560 rpm, but not below that, where the back-EMF signal is too small.